Informe del Diseño Arquitectónico del Sistema: Sazón Urbano

Nombres (EQUIPO 3):	**Matrícula**:
Cristopher Alan Pulgarin Santa Cruz	2105894
Francisco Sebastián Hernández Terán	2115798
Jesús Orlando Moreno Chávez	2108300

Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 1	**Actividad**: UNIDAD 3 - DISEÑO ARQUITECTÓNICO DEL SISTEMA
Fecha: 16/10/2026	

Bibliografía

1. Patrones de Lógica de Negocio, Service Layer y Unit of Work

- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

2. Domain-Driven Design (DDD), Agregados y Consistencia Eventual

- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.
- Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

3. CQRS (Command Query Responsibility Segregation)

- Young, G. (2010). *CQRS documents*. Greg Young Consulting. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

4. Principios SOLID y Arquitectura Limpia

- Martin, R. C. (2017). *Arquitectura limpia: Guía para especialistas en la estructura y el diseño de software*. Anaya Multimedia.

5. Registros de Decisiones de Arquitectura (ADR)

- Richards, M. y Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

6. Diagramas UML (Clases y Secuencia)

- Booch, G., Rumbaugh, J. y Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2.ª ed.). Addison-Wesley.

Índice

1. Alcance de la Arquitectura 3
   1.1 Delimitación del proceso de negocio 3
   1.2 Actores del sistema 3
   1.3 Casos de uso operativos seleccionados 3
2. Comparación y Selección del Patrón de Lógica de Negocio 4
3. Mapa de Dominio 5
   3.1 Lenguaje ubicuo (Glosario) 5
   3.2 Fichas técnicas de Aggregates 6
      Aggregate Root operativo: Pedido 6
      Aggregate Root logístico: Inventario 6
4. Service Layer y CQRS 7
   4.1 Application Services y manejadores de eventos 7
   4.2 CQRS básico 8
5. Repository y Unit of Work 9
6. Revisión SOLID (Etapa 7) 10
7. Registros de Decisiones de Arquitectura (ADR) 12
   7.1 ADR-001: Selección del patrón de lógica de negocio: Domain Model 12
   7.2 ADR-002: Consistencia transaccional eventual y Pedido como Aggregate Root 13
   7.3 ADR-003: CQRS básico y Application Services delgados 14
   7.4 ADR-004: Control de acceso (RBAC) mediante interceptores 15
   7.5 ADR-005: Historial inmutable de auditoría (Append-Only) 16
8. Diagramas de la Arquitectura 17
   8.1 Qué muestran los diagramas y cómo leerlos 20
   8.2 Diagrama 1: CU-01 Registrar Pedido (flujo exitoso) 21
      8.3 Diagrama 2: CU-03 Cancelar Pedido (flujo alternativo con consistencia eventual) 22
9. Validación de la Propuesta 23
10. Conclusiones de la Capa de Aplicación 25
   Anexo A. Matriz de Trazabilidad (Entregable 04) 25
      A.1 Necesidades de negocio hacia componentes 25
      A.2 Historias de usuario hacia casos de uso y componentes 27
   Anexo B. Bitácora de IA (Entregable 05) 29
   Anexo C. Código Base en C# (Entregable 06) 31
      C.1 Estructura de carpetas 31
      C.2 SazonUrbano.Domain 31
      C.3 SazonUrbano.Application 33
   Anexo D: Matriz de Trazabilidad Arquitectónica (Entregable 04) 34
      Dictamen de Cobertura 35

1. Alcance de la Arquitectura

1.1 Delimitación del proceso de negocio

El diseño arquitectónico formaliza el proceso operativo de **Sazón Urbano**, abarcando desde la recepción de pedidos omnicanal (mesa, teléfono, WhatsApp, domicilio), el procesamiento distribuido en estaciones de cocina, hasta la entrega final y el descuento asíncrono de ingredientes en el inventario. El alcance es el aprobado en el BRD de la Unidad 2; no se incorporaron necesidades ni reglas nuevas.

1.2 Actores del sistema

Actor	Responsabilidad
Gerente	Supervisa el funcionamiento general y requiere información para tomar decisiones.
Mesero/Caja	Realiza pedidos, solicita y realiza las operaciones en la caja.
Cocina	Consulta los pedidos y prepara los productos.
Encargado	Supervisa la operación y el control del inventario.
Repartidor	Recibe y entrega los pedidos a domicilio.
Cliente	Realiza pedidos, solicita modificaciones y recibe el producto.

1.3 Casos de uso operativos seleccionados

Se seleccionaron cinco casos de uso del proceso principal (pedido, preparación, entrega e inventario). **CU-01** constituye el flujo normal de escritura; **CU-03** incluye el flujo con excepción o rechazo (el intento de cancelación por un rol sin permiso es rechazado por el control de acceso). Todos tienen origen en las historias de usuario (US) y necesidades de negocio (NB) de la Unidad 2.

ID	Caso de uso	Actor	Reglas y escenarios relacionados	Resultado esperado
CU-01	Registrar Pedido	Mesero/Caja	NB-03, NB-06; US-08, US-09	Registra productos, cantidades y modificaciones.
CU-02	Procesar Pedido	Cocina	NB-03, NB-06; US-01, US-06	Visualiza comandas y avanza el estado

ID	Caso de uso	Actor	Reglas y escenarios relacionados	Resultado esperado
				(«En preparación», «Listo»).
CU-03	Cancelar Pedido	Mesero y Encargado	NB-05; US-10, US-11; ADR-004	Se solicita y luego se autoriza la cancelación.
CU-04	Entregar Pedido	Repartidor o Mesero	NB-06, NB-08; US-12	Marca la entrega, detonando la reducción de inventario.
CU-05	Actualizar Inventario y Mermas	Encargado	NB-01, NB-02, NB-07; US-03, US-04, US-05	Ajusta stock e identifica ingredientes críticos.

2. Comparación y Selección del Patrón de Lógica de Negocio

Se compararon **Transaction Script**, **Table Module** y **Domain Model** con criterios propios del caso. Para Sazón Urbano se adopta **Domain Model** (ADR-001): el sistema posee reglas de alta complejidad transaccional, como la verificación de inventario crítico en tiempo real y la gestión de «Mermas», así como estados de preparación concurrentes.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas (stock crítico, modificaciones, cancelación autorizada)	Simple al inicio, pero las reglas de inventario crítico y validación de modificaciones terminarían acopladas en funciones gigantes.	Adecuado para tablas estáticas; las reglas no se concentran en una sola tabla.	Encapsula comportamiento y estados cerca de las entidades de negocio.	NB-02; US-04, US-05, US-09, US-10
Cambios esperados (nuevas reacciones tras entregar un	Cada reacción nueva obliga a modificar el script de entrega	Cambios dispersos por módulo de tabla.	Se agregan manejadores de Domain Events sin editar el caso de	US-12; NB-07

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
pedido)	existente.		uso original.	
Interacción entre conceptos (Pedido, Inventario, Auditoría)	La validación de disponibilidad se duplicaría en múltiples controladores.	Dependencias fuertes que atraviesan varias tablas no encajan en un módulo por tabla.	Los agregados Pedido e Inventario mutan su propio estado con alta cohesión y bajo acoplamiento.	NB-01, NB-02; US-02
Estados e invariantes (Registrado, En preparación, Listo, Entregado, Cancelado)	Invariantes repartidas entre procedimientos.	Sin un lugar natural para proteger el ciclo de vida.	La entidad Pedido controla su ciclo de vida y sus invariantes.	NB-06; US-06, US-07, US-12
Trazabilidad de modificaciones y cancelaciones	Registro manual en cada procedimiento.	Registro disperso por tabla.	Centralizada mediante eventos de dominio.	NB-04, NB-05; US-09, US-10, US-13

Decisión y costo aceptado. Se elige Domain Model como patrón principal, sin combinación con otros patrones en la capa de negocio; las consultas de lectura masiva se resuelven fuera de los agregados mediante CQRS básico (sección 4). La consecuencia negativa aceptada es una mayor curva de aprendizaje, la necesidad de definir muy bien los Aggregate Roots y la creación de más clases e interfaces para separar el dominio de la persistencia.

3. Mapa de Dominio

3.1 Lenguaje ubicuo (Glosario)

Término	Definición
Comanda	Documento que recibe cocina detallando productos y modificaciones por estación.
Stock crítico	Nivel mínimo de existencia de un ingrediente que activa una alerta (US-05).
Merma	Diferencia del inventario por robo, error o desperdicio.
Trazabilidad	Registro inmutable (Auditoría) del 100% de las acciones (modificaciones, cancelaciones).

Término	Definición
Pedido	Agrupa productos y modificaciones; recorre los estados «Registrado», «En preparación», «Listo», «Entregado» y «Cancelado».
Inventario	Conjunto de existencias de ingredientes que se descuenta al entregar un pedido.
Modificación	Cambio o extra solicitado por el cliente sobre un producto del pedido.
LogAuditoria	Registro de solo anexado (Append-Only) con fecha, hora, usuario y motivo de cada acción trazable.

3.2 Fichas técnicas de Aggregates

Aggregate Root operativo: Pedido

- **Componentes:** ProductoPedido (Entity), Modificacion (Value Object).
- **Responsabilidades:** transicionar estados («Registrado», «En preparación», «Listo», «Entregado», «Cancelado») y validar las reglas de modificaciones y cancelaciones únicamente dentro del agregado.
- **Invariantes:** el pedido se crea con información completa (NB-03) y estado «Registrado» (NB-06); la cancelación registra motivo, y la autorización se verifica antes de entrar al dominio (ADR-004).
- **Domain Events:** PedidoRegistrado, PedidoEntregado, PedidoCancelado.

Aggregate Root logístico: Inventario

- **Componentes:** Ingrediente (Entity), AjusteMerma (Entity).
- **Responsabilidades:** descontar existencias, registrar mermas y evaluar alertas de stock crítico (NB-02).
- **Domain Events:** StockCriticoAlcanzado, InventarioActualizado.

Decisiones dudosas o descartadas. (a) *Inventario como raíz global*: descartada por problemas de concurrencia e impacto de rendimiento. (b) *Producto como centro*: incompatible con el ciclo de vida real, que pertenece al pedido completo. (c) *Pedido e Inventario en un mismo agregado (consistencia fuerte)*: descartada porque bloquea tablas durante el descuento. (d) *LogAuditoria*: no se modela como agregado con comportamiento, sino como registro de solo anexado. (e) *Domain Services*: no se identificó comportamiento que no pertenezca naturalmente a Pedido o Inventario; se deja pendiente de validación si surgen reglas entre agregados.

4. Service Layer y CQRS

Siguiendo la decisión de consistencia eventual, los *Application Services* fungen puramente como orquestadores: reciben el comando, cargan el agregado, ejecutan el método de dominio y cierran la transacción con el Unit of Work.

4.1 Application Services y manejadores de eventos

Servicio	Caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
RegistrarPedidoHandler	CU-01	RegistrarPedidoCommand: canal, productos, modificaciones, usuario	ControlAccesoRBAC, IPedidoRepository, IUnitOfWork, IDespachadorEventos	Result.exito con el ID del pedido	Validación de datos iniciales y estado «Registrado» al agregado Pedido (NB-03, NB-06)
Handler de avance de estado (nombre por definir en código)	CU-02	ID de pedido y nuevo estado	IPedidoRepository, IUnitOfWork	Result	Transiciones permitidas al agregado Pedido (US-06)
CancelarPedidoHandler	CU-03	CancelarPedidoCommand: ID de pedido, motivo, usuario	ControlAccesoRBAC, IPedidoRepository, IUnitOfWork, IDespachadorEventos	Result de éxito o rechazo	Cambio a «Cancelado» al agregado Pedido; permiso al interceptor RBAC (ADR-004)
EntregarPedidoHandler	CU-04	ID de pedido, usuario	IPedidoRepository, IUnitOfWork, IDespachadorEventos	Result	Cambio a «Entregado» al agregado Pedido; publica PedidoEntregado
DescontarInventarioHandler (manejador de	CU-04 (reacción)	Evento PedidoEntregado	IInventarioRepository, IUnitOfWork	Sin respuesta al usuario	Descuento de existencias y evaluación de stock crítico al

Servicio	Caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
evento)					agregado Inventario
ManejadorPedidoCancelado (manejador de evento)	CU-03 (reacción)	Evento PedidoCancelado (usuario, motivo, hora)	IRegistroAuditoriaRepository, IUnitOfWork	Sin respuesta al usuario	Inmutabilidad garantizada por el contrato append-only (ADR-005)
Handler de ActualizarStockCommand	CU-05	Entradas o mermas físicas	IInventarioRepository, IUnitOfWork	Result	Stock y alertas al agregado Inventario

Decisión clave (descuento de inventario). Según el BRD, los ingredientes se restan al momento de entregar el pedido. Por tanto, EntregarPedidoHandler modifica el estado del Pedido a «Entregado» y despacha el evento PedidoEntregado. DescontarInventarioHandler escucha el evento y, en una transacción totalmente separada y asíncrona, carga el agregado Inventario para deducir las existencias, garantizando consistencia sin bloquear la operación de la caja o del repartidor.

Revisión obligatoria. Los nombres de los métodos expresan acciones del negocio; los handlers no evalúan reglas, solo coordinan; dependen únicamente de interfaces; cada operación tiene un inicio y un final transaccional definidos; y los errores del dominio se expresan con términos del negocio.

4.2 CQRS básico

Dado que Sazón Urbano exige vistas complejas que no deben bloquear operaciones, se separan **lógicamente** lecturas de escrituras (sin bases de datos separadas).

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Command	RegistrarPedidoCommand	Canal, productos, modificaciones, usuario	RegistrarPedidoHandler	ID del pedido	Alta transaccionalidad en caja (US-08, US-09).
Command	CancelarPedidoCommand	ID de pedido, motivo; requiere autorización del Encargado	CancelarPedidoHandler	Éxito o rechazo	Operación crítica con RBAC y auditoría (US-10).

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Command	ActualizarStockCommand	Ingrediente, cantidad, tipo (entrada o merma)	Handler de ActualizarStockCommand	Confirmación	Registra entradas o mermas físicas (US-03).
Query	ConsultarDisponibilidadQuery	Producto (búsqueda por nombre)	Query Handler asociado	ViewModel de disponibilidad y stock	El Mesero verifica stock rápido para evitar vender agotados (US-02).
Query	ConsultarPedidoCocinaQuery	ID de pedido	Query Handler asociado	ViewModel separado por estaciones de cocina	Muestra productos, cantidades, modificaciones y mesa (US-01).
Query	ConsultarReportesQuery	Tipo de reporte, período	Query Handler asociado	ViewModel plano de lectura masiva	Uso exclusivo del Gerente para cruzar ventas e inventario (US-14, US-15).

Las Queries devuelven ViewModels sin usar Agregados, de modo que el modelo de lectura no se reutiliza como modelo de escritura. No se separa físicamente el almacenamiento: la separación es lógica, en la capa de aplicación.

5. Repository y Unit of Work

Se definen abstracciones en términos del dominio para evitar acoplamiento a Entity Framework. La implementación concreta pertenece a la capa de Infraestructura.

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
IPedidoRepository	Cargar y persistir el agregado Pedido; listar pedidos para cocina	obtenerPorId(), guardar(), listarPendientesCocina()	Pedido

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
IInventarioRepository	Cargar y persistir el agregado Inventario	obtenerPorIngrediente(), guardar()	Inventario
IRegistroAuditoriaRepository	Trazabilidad inmutable	agregar() (único método)	LogAuditoria

IUnitOfWork coordina commit y rollback bajo la regla **una transacción por Aggregate Root**.

- **Inicio:** al comenzar el handler, antes de mutar el agregado.
- **Cambios incluidos:** únicamente el estado de un agregado y sus entidades internas (por ejemplo, Pedido con sus ProductoPedido).
- **Error:** si ocurre una falla al guardar, la transacción se revierte y el evento de dominio nunca se despacha, manteniendo la integridad.
- **Entre agregados:** las interacciones (descontar inventario al entregar; auditar al cancelar) ocurren en transacciones separadas disparadas por Domain Events.

6. Revisión SOLID (Etapa 7)

Se evaluó la arquitectura con los cinco principios, enfocada en la interacción entre las capas de Dominio, Aplicación e Infraestructura. Cada hallazgo indica evidencia, impacto y cambio realizado.

Principio	Hallazgo / evidencia de riesgo (antipatrón)	Impacto	Corrección y mitigación aplicada en el diseño
S (Single Responsibility)	Existía el riesgo de que los Handlers (por ejemplo, RegistrarPedidoHandler) evaluaran reglas de negocio, o de que entidades como Pedido ejecutaran operaciones directas de guardado.	Responsabilidades mezcladas; varias razones para cambiar una misma clase.	Separación estricta: Pedido e Inventario (Dominio) son los únicos responsables de mutar y validar su estado; los Handlers (Aplicación) solo orquestan; los repositorios (Infraestructura) solo persisten.
O (Open/Closed)	Agregar reacciones secundarias tras entregar un pedido (enviar un correo o actualizar el inventario) requeriría modificar el código existente del	Cada nueva política obliga a editar el orquestador original.	Se implementaron Domain Events (por ejemplo, PedidoEntregadoEvent). El sistema está abierto a la extensión mediante nuevos manejadores

Principio	Hallazgo / evidencia de riesgo (antipatrón)	Impacto	Corrección y mitigación aplicada en el diseño
	caso de uso de entrega.		(como DescontarInventarioHandler) y cerrado a la modificación de los orquestadores originales.
L (Liskov Substitution)	Incompatibilidad potencial al sustituir repositorios simulados en memoria (InMemory) por implementaciones reales sobre SQL Server o Entity Framework Core.	Fallos al cambiar de implementación; pruebas no representativas.	Las clases concretas de infraestructura implementan estrictamente las firmas de IPedidoRepository e IInventarioRepository sin alterar el comportamiento esperado por la capa de aplicación, garantizando la sustitución sin fallos.
I (Interface Segregation)	Riesgo de una interfaz monolítica de base de datos que obligara a la tabla de auditoría a tener métodos de edición o borrado que nunca va a utilizar.	Posibilidad de actualizar o borrar un log de auditoría.	Contratos segregados por Aggregate Root. IRegistroAuditoriaRepository expone exclusivamente agregar(), impidiendo arquitectónicamente que se actualice o borre un registro de auditoría.
D (Dependency Inversion)	La capa de aplicación instanciaba directamente clases concretas de bases de datos (por ejemplo, new SqlPedidoRepository()).	Acoplamiento de la lógica del restaurante al motor de datos.	Los Application Services dependen exclusivamente de abstracciones (IUnitOfWork, IPedidoRepository) inyectadas. El Dominio ignora por completo la existencia de la base de datos.

Observación complementaria (sexto hallazgo, capa de aplicación). Al aplicar CQRS básico se reforzó el principio de Responsabilidad Única: las lecturas masivas del Gerente para cruzar ventas e inventario (US-14, US-15) están encapsuladas en Query Handlers (ConsultarReportesQuery), cuya única responsabilidad es leer y proyectar datos hacia la interfaz (vistas planas), sin sobrecargar la lógica transaccional de los Command Handlers que utilizan los meseros y la cocina.

7. Registros de Decisiones de Arquitectura (ADR)

Se documentan cinco ADR con el formato de Richards y Ford (2020): estado, contexto, alternativas, decisión, consecuencias y trazabilidad.

7.1 ADR-001: Selección del patrón de lógica de negocio: Domain Model

Campo	Contenido
Estado	Aceptada (15 de octubre de 2026).
Contexto	Sazón Urbano requiere gestionar la recepción de pedidos por diversos canales, el control de existencias de ingredientes, modificaciones, cancelaciones y actualizaciones de estado de preparación. El caso presenta reglas de negocio que interactúan fuertemente con el ciclo de vida del pedido.
Alternativas	1. Transaction Script: simple, pero las reglas de inventario crítico y validación de modificaciones terminarían acopladas en funciones gigantes. 2. Table Module: útil para tablas estáticas, pero Sazón Urbano tiene dependencias fuertes que atraviesan varias tablas. 3. Domain Model: encapsula el comportamiento y los estados cerca de las entidades de negocio.
Decisión	Se adopta Domain Model como patrón principal. La entidad Pedido controlará su ciclo de vida y sus invariantes (modificaciones, estados). Las validaciones operativas puras se delegan a las entidades en memoria antes de la persistencia.
Consecuencias	Positivas: lógica centralizada de estados del pedido; facilita el cumplimiento de la trazabilidad requerida en modificaciones. Costos: mayor curva de aprendizaje, requiere definir muy bien los Aggregate Roots y crear más clases e interfaces para separar el dominio de la persistencia.

Campo	Contenido
Trazabilidad	NB-01, NB-02, NB-06; US-02, US-04, US-05, US-06, US-08, US-09, US-10.

7.2 ADR-002: Consistencia transaccional eventual y Pedido como Aggregate Root

Campo	Contenido
Estado	Aceptada (15 de octubre de 2026).
Contexto	El proceso central del negocio gira alrededor del pedido. Un pedido agrupa productos y modificaciones, y al concretar una entrega, el inventario debe descontarse con precisión.
Alternativas	1. Inventario como raíz global: evita sobreventas, pero causaría problemas de concurrencia e impacto de rendimiento masivo. 2. Producto como centro: incompatible con el ciclo de vida real que pertenece al pedido completo. 3. Pedido como Aggregate Root (consistencia fuerte): bloquea las tablas durante el descuento. 4. Pedido e Inventario separados (consistencia eventual): elegida para maximizar el rendimiento.
Decisión	La entidad Pedido será un Aggregate Root independiente que manejará su transición de estado («Registrado», «Entregado», etc.). Para evitar cuellos de botella, se rechaza el descuento en una misma transacción. El Unit of Work confirmará la entrega del pedido y despachará el evento PedidoEntregadoEvent. Un manejador asíncrono actualizará el Inventario en una transacción completamente separada.
Consecuencias	El límite de consistencia del Pedido queda claro, evitando bloquear todo el sistema cuando se preparan diferentes pedidos. Las reglas de modificaciones o cancelaciones se validan únicamente dentro del agregado Pedido. Existe un mínimo desfase de milisegundos en la vista del inventario.
Trazabilidad	NB-04, NB-05, NB-07; US-06, US-07, US-10, US-12.

7.3 ADR-003: CQRS básico y Application Services delgados

Campo	Contenido
Estado	Aceptada (15 de octubre de 2026).
Contexto	Sazón Urbano tiene dos necesidades distintas: alta transaccionalidad de escritura por parte de meseros y cocina (crear pedidos, actualizar estados), y consultas de historial y cruces de ventas por parte del gerente y encargado.
Alternativas	1. Arquitectura tradicional N-Capas: acopla los modelos de lectura y escritura, haciendo las consultas lentas al cargar entidades pesadas. 2. CQRS básico: elegida para aislar las intenciones de lectura y escritura.
Decisión	Aplicar CQRS básico: Commands para acciones de escritura (por ejemplo, RegistrarPedidoCommand) y Queries para lectura (por ejemplo, ConsultarDisponibilidadQuery). Los Application Services serán delgados: solo recibirán el comando, cargarán el agregado, ejecutarán el método de dominio y cerrarán la transacción con el UnitOfWork. Las lecturas devolverán directamente ViewModels.
Consecuencias	Positivas: los reportes de ventas e inventario que requiere el gerente se procesan rápidamente sin interferir con las operaciones de la caja y los meseros. Costos: se requiere implementar un número mayor de Handlers y DTOs para separar la lectura de la escritura.
Trazabilidad	NB-03, NB-09; US-01, US-11, US-13, US-14, US-15.

7.4 ADR-004: Control de acceso (RBAC) mediante interceptores

Campo	Contenido
Estado	Aceptada (15 de octubre de 2026).
Contexto	Sazón Urbano requiere restringir ciertas operaciones críticas. Específicamente, solo el Encargado o Gerente tiene la facultad de autorizar la cancelación de un pedido, evitando que un Mesero lo haga unilateralmente.
Alternativas	1. Validación en la capa de Dominio: descartada porque obligaría a inyectar datos de sesión (usuario, rol) en las entidades, mezclando seguridad con reglas comerciales. 2. Validación en la Service Layer (elegida): utilizar un middleware o interceptor antes de la ejecución del comando.
Decisión	El control de acceso basado en roles (RBAC) se manejará mediante un interceptor en la capa de aplicación. Antes de que se ejecute CancelarPedidoHandler, el interceptor validará el rol del usuario. La capa de Dominio permanecerá totalmente agnóstica a las sesiones de los usuarios.
Consecuencias	Positivas: respeta el Principio de Responsabilidad Única y mantiene el modelo de dominio limpio de dependencias de infraestructura. Costos: requiere que la identidad del usuario (userId, rol) se extraiga en la capa de aplicación y viaje como parámetro hasta los eventos de dominio para poder guardarse en la auditoría.
Trazabilidad	US-10 (registrar cancelación autorizada por encargado).

7.5 ADR-005: Historial inmutable de auditoría (Append-Only)

Campo	Contenido
Estado	Aceptada (15 de octubre de 2026).
Contexto	Uno de los tres problemas más urgentes del restaurante es la dificultad para rastrear modificaciones y cancelaciones. Se requiere una trazabilidad absoluta donde el 100% de las acciones queden registradas con fecha, hora, usuario y motivo para auditoría.
Alternativas	1. Sobrescribir registros (Update): descartada, ya que se pierde el historial de los cambios anteriores. 2. Event Sourcing: descartada por ser una sobreingeniería excesiva para el alcance de este proyecto escolar. 3. Tablas Append-Only relacionales (elegida): un punto intermedio efectivo y simple.
Decisión	Se implementará un LogAuditoria en una tabla de solo anexado (Append-Only). Las operaciones que muten el estado (como modificaciones o cancelaciones) emitirán eventos de dominio. Un Event Handler asíncrono capturará estos eventos e insertará un nuevo registro en la tabla de auditoría, sin modificar ni borrar jamás los registros previos.
Consecuencias	Positivas: resuelve la urgencia de trazabilidad del negocio y proporciona datos precisos para que el gerente detecte anomalías (NB-09). Costos: aumenta ligeramente el uso de almacenamiento en la base de datos a largo plazo.
Trazabilidad	NB-04, NB-05, NB-09; US-09, US-10, US-11, US-13.

8. Diagramas de la Arquitectura

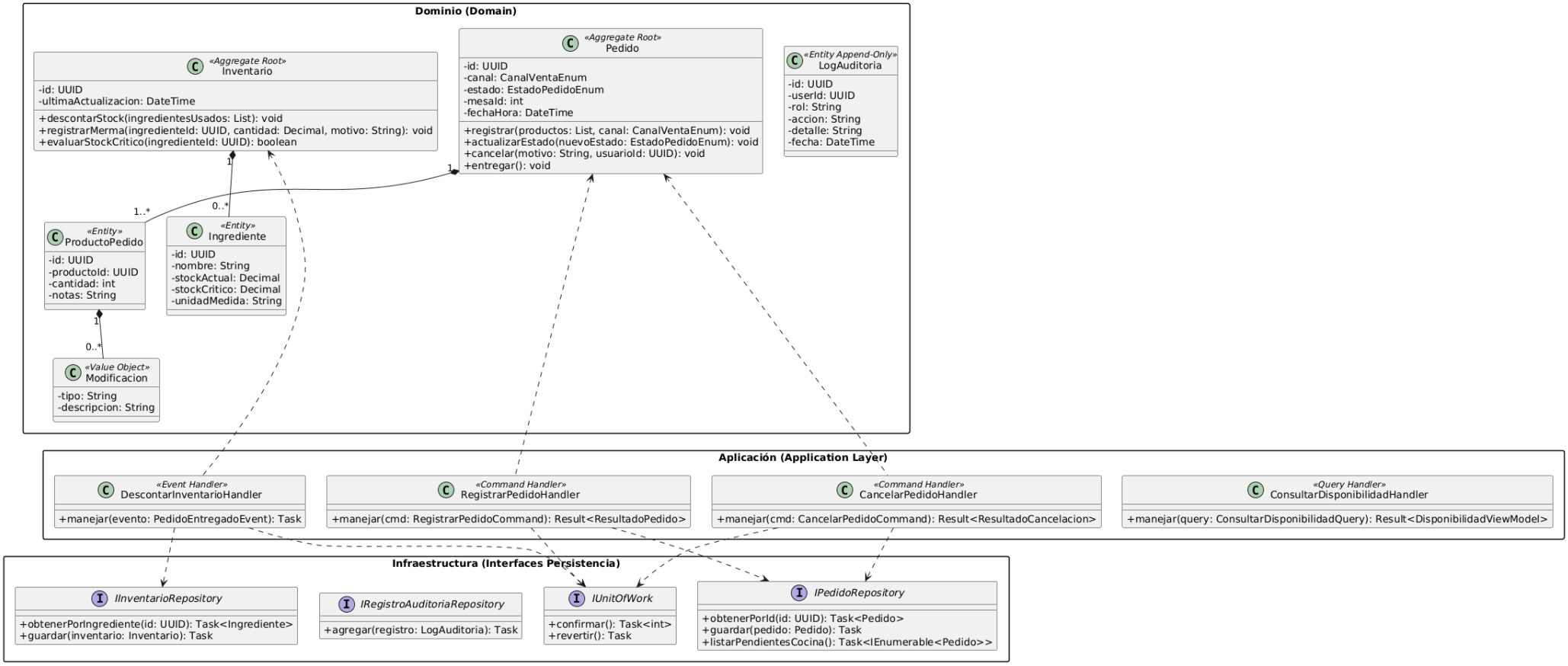


Diagrama 1: Flujo Exitoso - CU-01 Registrar Pedido (US-08)

Este diagrama muestra el flujo normal donde el Mesero registra un pedido, aplicando una transacción única y emitiendo un evento.

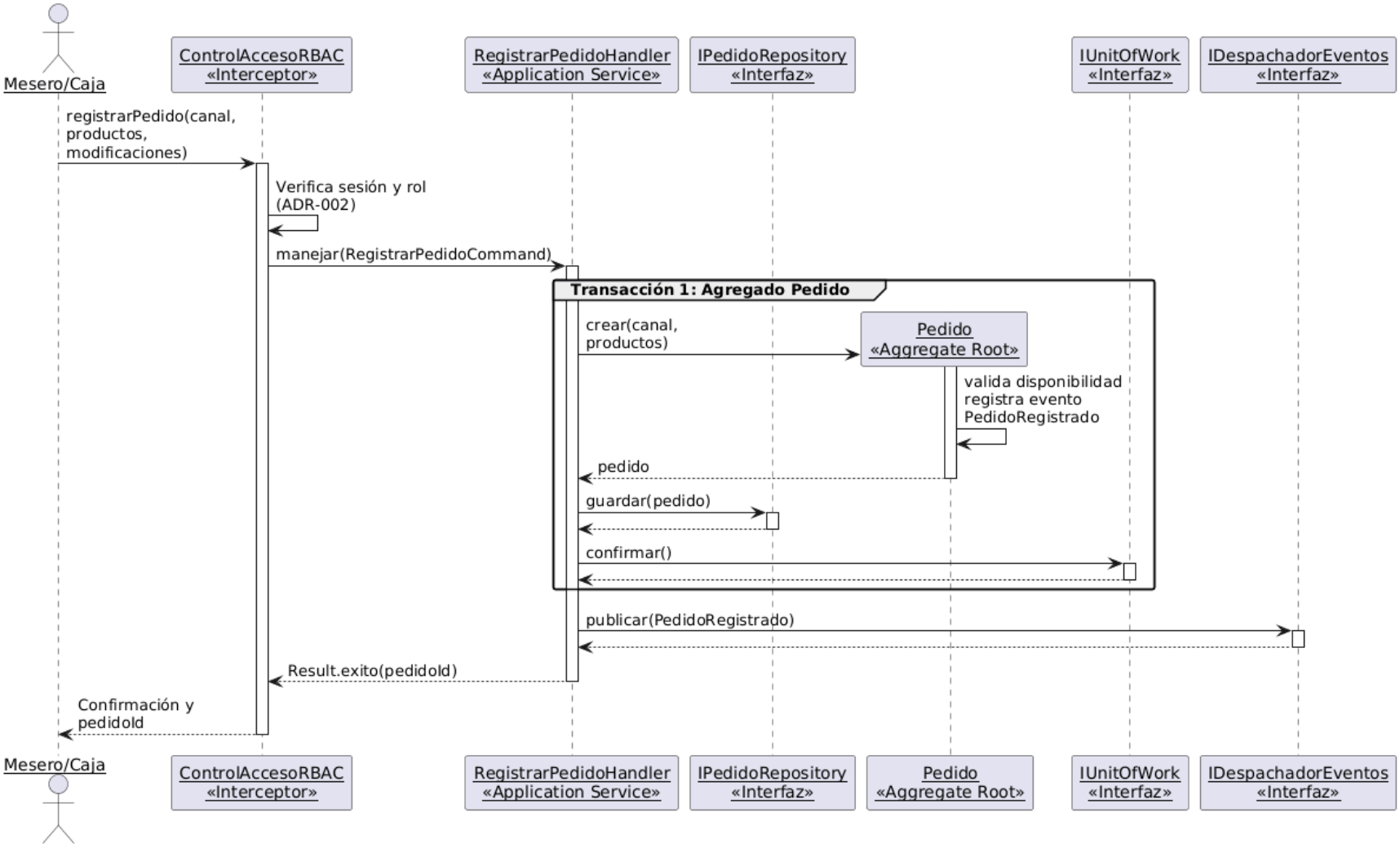
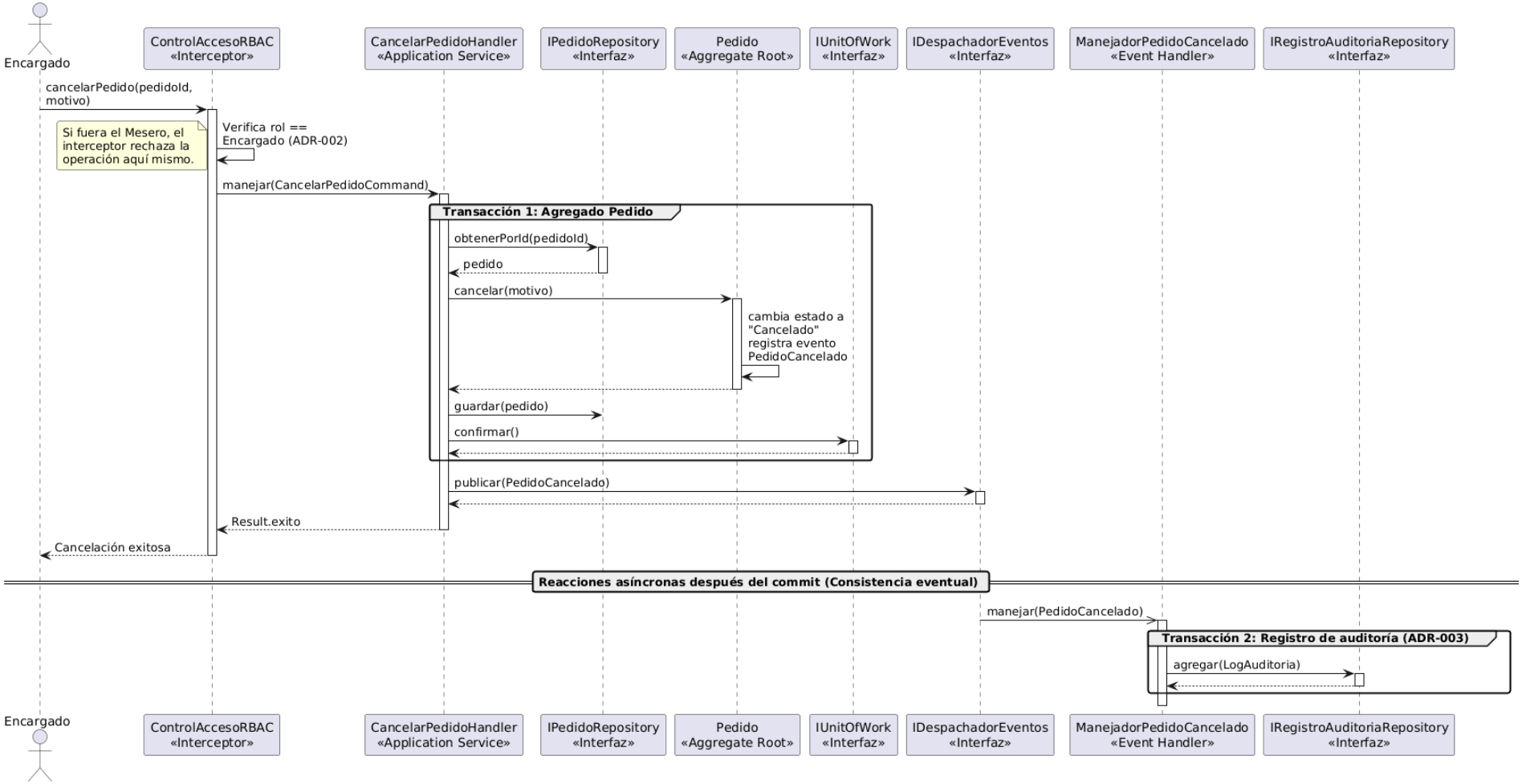


Diagrama 2: Flujo Alternativo - CU-03 Cancelar Pedido (US-10)

Este diagrama es clave porque demuestra las decisiones arquitectónicas que tomaste: el control de acceso (ADR-002) donde solo el Encargado puede cancelar, y la consistencia eventual para registrar la auditoría inmutable (ADR-003).



8.1 Qué muestran los diagramas y cómo leerlos

Participante	Tipo	Responsabilidad	Referencia
Mesero/Caja / Encargado	Actor	Inicia la operación y recibe el resultado.	CU-01, CU-03
ControlAccesoRBAC	Interceptor	Verifica el rol del usuario antes de entrar a la Service Layer. No interviene el dominio.	ADR-004
RegistrarPedidoHandler / CancelarPedidoHandler	Application Service	Coordina los casos de uso, actuando como orquestador sin evaluar reglas de negocio.	Sección 4
IPedidoRepository	Interfaz	Abstrae la persistencia para cargar y guardar el agregado Pedido.	Sección 5
Pedido	Aggregate Root	Aplica las reglas de negocio, muta su estado y registra los eventos de dominio.	Mapa de dominio
IUnitOfWork	Interfaz	Coordina y confirma la transacción atómica.	Sección 5
IDespachadorEventos	Interfaz	Entrega los eventos en memoria a los suscriptores después del commit.	Sección 4
ManejadorPedidoCancelado	Event Handler	Escucha la cancelación y orquesta la escritura en auditoría.	ADR-005
IRegistroAuditoriaRepository	Interfaz	Abstracción que solo permite agregar registros (Append-Only) para trazabilidad.	ADR-005

Los dos diagramas de secuencia representan la arquitectura descrita en los capítulos anteriores, basándose en CQRS y Domain Model. El primero cubre un flujo operativo exitoso de escritura (CU-01 / US-08) y el segundo ilustra un flujo con validación de seguridad y consistencia eventual para trazabilidad (CU-03 / US-10).

Para facilitar la lectura de los diagramas generados en PlantUML se utilizan dos convenciones: los fragmentos etiquetados como «Transacción 1» o «Transacción 2» encierran las operaciones que se confirman de una sola vez en la base de datos (atomicidad); las flechas con punta abierta (o flujos asíncronos después del commit) indican procesos donde el emisor no espera la respuesta para continuar.

8.2 Diagrama 1: CU-01 Registrar Pedido (flujo exitoso)

Campo	Contenido
Identificador	Secuencia_US08_RegistrarPedido
Caso de uso	CU-01 Registrar pedido (US-08). Actor: Mesero/Caja.
Regla	NB-03 (información completa) y NB-06 (estado del pedido).
Operación	RegistrarPedidoCommand procesado en RegistrarPedidoHandler.
Objetivo	Mostrar cómo un pedido se registra de forma atómica sobre el agregado Pedido y cómo se emite un evento a memoria para futuras integraciones.

Pasos	Qué ocurre
1 a 3	El Mesero envía la solicitud con el canal, productos y modificaciones. El interceptor ControlAccesoRBAC verifica la sesión y delega el comando al RegistrarPedidoHandler.
4 a 7	Inicia la Transacción 1. El handler solicita a la entidad Pedido su creación. El agregado valida los datos iniciales, establece su estado como «Registrado» y acumula internamente el evento PedidoRegistrado.
8 a 13	El handler invoca a IPedidoRepository para guardar el nuevo estado. Inmediatamente después, llama a IUnitOfWork.confirmar() para hacer commit en la base de datos y cerrar la Transacción 1.
14 a 17	Una vez confirmada la persistencia, el handler utiliza IDespachadorEventos para publicar el

Pasos	Qué ocurre
	evento en memoria y retorna un Result.exito con el ID del pedido al Mesero, a través del interceptor.

Límites transaccionales. Existe una única transacción que protege exclusivamente al Aggregate Root Pedido. Si ocurre algún fallo al guardar en el repositorio, la transacción se revierte y el evento de dominio nunca es despachado, manteniendo la integridad del sistema.

8.3 Diagrama 2: CU-03 Cancelar Pedido (flujo alternativo con consistencia eventual)

Campo	Contenido
Identificador	Secuencia_US10_CancelarPedido
Caso de uso	CU-03 Cancelar pedido (US-10). Actor: Encargado.
Reglas	NB-05 (seguimiento a cancelaciones), ADR-004 (RBAC) y ADR-005 (auditoría inmutable).
Operación	CancelarPedidoCommand procesado en CancelarPedidoHandler.
Objetivo	Mostrar el bloqueo de seguridad (RBAC) antes del dominio y la reacción asíncrona para cumplir con la trazabilidad mediante consistencia eventual.

Pasos	Qué ocurre
1 a 3	El Encargado solicita la cancelación. ControlAccesoRBAC intercepta la llamada y verifica que el rol tenga permisos. (Si un Mesero intentara esta acción, el flujo sería rechazado aquí mismo.) Al ser válido, pasa al handler.
4 a 9	Inicia la Transacción 1. El handler carga el agregado Pedido desde el repositorio usando el ID e invoca cancelar(motivo) del dominio, que cambia el estado a «Cancelado» y registra el evento PedidoCancelado.
10 a 14	El handler guarda los cambios mediante el repositorio, confirma la Transacción 1 con IUnitOfWork y publica el evento PedidoCancelado en el despachador.
15 a 17	El handler devuelve la respuesta de éxito al

Pasos	Qué ocurre
	Encargado. La interacción con el usuario termina aquí.
18 a 22	**Reacción asíncrona:** ManejadorPedidoCancelado recibe el evento. Inicia la **Transacción 2**, crea un registro inmutable de la acción (con usuario, motivo y hora) y lo envía a IRegistroAuditoriaRepository.agregar(), cumpliendo la exigencia de trazabilidad sin bloquear la cancelación original.

Límites transaccionales y consistencia eventual. Este escenario demuestra la aplicación estricta del ADR-005. Existen dos transacciones separadas para respetar el límite de los agregados: la primera protege la mutación del Pedido; la segunda, una vez confirmada la primera, escribe exclusivamente en la tabla Append-Only de auditoría. Si el sistema cayera después de la Transacción 1, el pedido quedaría cancelado pero el registro de auditoría podría requerir mecanismos de compensación, lo cual es un riesgo aceptado a cambio de un mayor rendimiento.

9. Validación de la Propuesta

Antes del cierre del documento, el equipo realizó una revisión integral de la arquitectura, recorriendo los casos de uso operativos sobre los diagramas UML, los diagramas de secuencia y los ADR. La siguiente tabla responde a las siete preguntas de control de la guía de la materia.

Validación	Pregunta oficial de la guía	Resultado y justificación del Equipo 3
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	**Cumple.** El 100% de los componentes arquitectónicos (agregados, interfaces de repositorio y Handlers de CQRS) trazan directamente a las Necesidades del Negocio (NB-01 a NB-09) y a las Historias de Usuario (US-01 a US-15) definidas en el BRD. Ver Anexo A.
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	**Cumple.** Se unificó y respetó el Lenguaje Ubicuo (Pedido, Inventario, Merma, Stock Crítico, Comanda, Modificación, LogAuditoria) en todas las especificaciones, diagramas

Validación	Pregunta oficial de la guía	Resultado y justificación del Equipo 3
		UML y decisiones.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	**Cumple.** Las invariantes del negocio (evaluación de stock crítico, transiciones de estado) viven encapsuladas en las entidades ricas (Pedido, Inventario) bajo el patrón Domain Model. La Service Layer actúa únicamente como orquestadora del flujo (CQRS), sin absorber ni evaluar lógica comercial.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	**Cumple.** Los participantes de los diagramas de secuencia (RegistrarPedidoHandler, ControlAccesoRBAC, IPedidoRepository, ManejadorPedidoCancelado) corresponden de manera exacta a las interfaces y clases modeladas en el Diagrama de Clases UML estructural.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	**Cumple.** El modelo de dominio es agnóstico de la persistencia tecnológica; consume contratos abstractos (IPedidoRepository, IInventarioRepository, IRegistroAuditoriaRepository) inyectados mediante Inversión de Dependencias (DIP).
Transacción	¿El límite de la operación y su manejo de error están definidos?	**Cumple.** Se delineó una única transacción atómica por cada Aggregate Root coordinada mediante IUnitOfWork. Las interacciones entre agregados distintos (por ejemplo, descontar inventario al entregar un pedido) se gestionan asíncronamente mediante

Validación	Pregunta oficial de la guía	Resultado y justificación del Equipo 3
		consistencia eventual (Domain Events).
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	**Cumple.** Cada abstracción, patrón y diagrama generado con asistencia de IA fue analizado y validado técnicamente por los integrantes humanos del equipo, garantizando que el diseño no introduzca antipatrones, alucinaciones ni desvíos de los requerimientos de Sazón Urbano. Ver Anexo B.

10. Conclusiones de la Capa de Aplicación

La adopción de consistencia eventual como decisión estructural (ADR-002) permitió delimitar con claridad la frontera transaccional de cada Aggregate Root. Al despachar el evento PedidoEntregado una vez confirmada la transacción del Pedido, y delegar el descuento del Inventario a un manejador asíncrono en una transacción independiente, se evitan bloqueos de base de datos en horas pico y se mantiene fluida la operación de la caja y del repartidor. El costo asumido es un desfase mínimo en la vista del inventario y la necesidad de mecanismos de compensación ante una caída del sistema entre transacciones, riesgo que el equipo acepta de forma explícita a cambio de mayor rendimiento y de un bajo acoplamiento entre los agregados.

Por su parte, CQRS básico (ADR-003) consolida el papel de los Application Services como orquestadores delgados: los Commands cambian el estado a través del dominio y de IUnitOfWork, mientras que las Queries devuelven ViewModels de lectura sin cargar agregados, lo que permite atender los reportes del Gerente (US-14, US-15) y las vistas de cocina (US-01) sin interferir con la alta transaccionalidad de la operación. La separación se mantiene lógica y no física, lo que limita la complejidad adicional a un mayor número de Handlers y DTOs. En conjunto, ambas decisiones fortalecen los principios SOLID evaluados (SRP, OCP y DIP) y dejan un diseño extensible, trazable y coherente con las necesidades del negocio.

Anexo A. Matriz de Trazabilidad (Entregable 04)

A.1 Necesidades de negocio hacia componentes

Necesidad	Descripción	Historias de usuario	Componentes arquitectónicos	ADR

Necesidad	Descripción	Historias de usuario	Componentes arquitectónicos	ADR
NB-01	Mantener actualizada la disponibilidad de productos	US-02, US-03	Agregado Inventario; ConsultarDisponibilidadQuery; Handler de ActualizarStockCommand	ADR-001
NB-02	Evitar venta de productos por falta de ingredientes	US-02, US-04, US-05	Inventario (evaluación de stock crítico); evento StockCriticoAlcanzado; ConsultarDisponibilidadQuery	ADR-001
NB-03	Asegurar que cocina reciba información completa	US-01, US-08, US-09	Pedido (validación inicial); RegistrarPedidoHandler; ConsultarPedidoCocinaQuery	ADR-003 (y ADR-001)
NB-04	Registrar modificaciones de clientes	US-09, US-13	Pedido (Modificacion como Value Object); ManejadorPedidoCancelado y manejadores de auditoría; LogAuditoria	ADR-002, ADR-005
NB-05	Registrar y dar seguimiento a cancelaciones	US-10, US-11	CancelarPedidoHandler; ControlAccesoRBAC; PedidoCancelado; ManejadorPedidoCancelado; IRegistroAuditoriaRepository	ADR-002, ADR-004, ADR-005

Necesidad	Descripción	Historias de usuario	Componentes arquitectónicos	ADR
NB-06	Conocer el estado de cada pedido	US-06, US-07, US-12	Pedido (transición de estados); EntregarPedidoHandler; handler de avance de estado	ADR-001, ADR-002
NB-07	Reducir diferencias entre ventas e inventario	US-03, US-04, US-14	PedidoEntregado; DescontarInventarioHandler; Inventario; ConsultarReportesQuery	ADR-002
NB-08	Control de pedidos para llevar y a domicilio	US-08, US-12	RegistrarPedidoHandler (canal); EntregarPedidoHandler	ADR-001
NB-09	Proporcionar información útil a gerencia y encargados	US-11, US-13, US-14, US-15	ConsultarReportesQuery; consultas sobre LogAuditoria; IRegistroAuditoriaRepository	ADR-003, ADR-005

A.2 Historias de usuario hacia casos de uso y componentes

Historia	Caso de uso	Componentes (capa)	ADR
US-01 Consultar pedido en cocina	CU-02	ConsultarPedidoCocinaQuery (Aplicación); IPedidoRepository.listarPendientesCocina()	ADR-003
US-02 Consultar disponibilidad	CU-01 (apoyo)	ConsultarDisponibilidadQuery (Aplicación)	ADR-001, ADR-003
US-03 Actualizar inventario	CU-05	Inventario (Dominio); ActualizarStockCommand; IInventarioRepository	ADR-001 (*)
US-04 Identificar ingredientes críticos	CU-05	Inventario; StockCriticoAlcanzado	ADR-001

Historia	Caso de uso	Componentes (capa)	ADR
US-05 Alertar sobre productos agotados	CU-05	Inventario; StockCriticoAlcanzado	ADR-001
US-06 Actualizar estado del pedido	CU-02	Pedido; handler de avance de estado	ADR-001, ADR-002
US-07 Consultar estado del pedido	CU-02	Pedido (estado); Query de estado (nombre por definir)	ADR-002
US-08 Registrar pedido	CU-01	RegistrarPedidoCommand; RegistrarPedidoHandler; Pedido; PedidoRegistrado	ADR-001
US-09 Registrar modificaciones	CU-01	Pedido; Modificacion (Value Object); LogAuditoria	ADR-001, ADR-005
US-10 Registrar cancelación	CU-03	CancelarPedidoCommand; CancelarPedidoHandler; ControlAccesoRBAC; Pedido; ManejadorPedidoCancelado	ADR-001, ADR-002, ADR-004, ADR-005
US-11 Consultar cancelaciones	CU-03 (consulta)	Query sobre LogAuditoria (nombre por definir)	ADR-003, ADR-005
US-12 Registrar entrega	CU-04	EntregarPedidoHandler; Pedido; PedidoEntregado; DescontarInventarioHandler	ADR-002
US-13 Consultar historial de modificaciones	CU-01 (consulta)	Query sobre LogAuditoria (nombre por definir)	ADR-003, ADR-005
US-14 Consultar ventas e inventario	N/A (lectura)	ConsultarReportesQuery	ADR-003
US-15 Generar	N/A (lectura)	ConsultarReportesQuer	ADR-003

Historia	Caso de uso	Componentes (capa)	ADR
reportes		y	

(*) US-03 no figura en la trazabilidad original de ningún ADR; se relaciona con ADR-001 por aplicarse Domain Model al agregado Inventario. Los nombres marcados como «por definir» corresponden a Queries y handlers que la Unidad 2 exige pero que aún no tienen nombre formal en el diseño; se registran como hallazgo pendiente de validación del equipo.

Anexo B. Bitácora de IA (Entregable 05)

Registro de las iteraciones realizadas con IA como copiloto. En todos los casos la decisión final fue del equipo, verificada contra los documentos de la Unidad 2.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	Comparar patrones de lógica de negocio	Con base en las reglas de inventario crítico, modificaciones, cancelaciones y estados del pedido de Sazón Urbano, compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.	Tabla comparativa con ventajas, riesgos y costos de cada patrón.	Supuso que la validación de disponibilidad era una regla simple, subestimando la interacción entre Pedido e Inventario.	Se adoptó Domain Model (ADR-001), ajustando los criterios a la evidencia de la Unidad 2 (NB-02, US-04, US-05, US-06, US-10).
2	Redactar el ADR-002 para el descuento de inventario	Le pedimos a la IA el ADR-002 para el descuento de inventario.	Estructura del ADR y alternativas sobre la raíz del agregado.	La IA propuso actualizar el Pedido y el Inventario en la misma transacción	Se corrigió forzando la Consistencia Eventual mediante Domain

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
				(consistencia fuerte). El equipo detectó el error arquitectónico porque bloquearía la base de datos en horas pico.	Events: EntregarPedidoHandler confirma el Pedido y despacha PedidoEntregado; DescontarInventarioHandler actualiza el Inventario en otra transacción.
3	Diseñar la Service Layer y el control de acceso para la cancelación	A partir de los casos de uso CU-01 a CU-05, propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.	Firmas de handlers y separación entre Commands y Queries.	Propuso validar el rol del usuario dentro del método cancelar() de Pedido, mezclando seguridad con reglas comerciales.	Se rechazó: el RBAC se resuelve con un interceptor en la capa de aplicación y el dominio permanece agnóstico a la sesión (ADR-004).
4	Revisión SOLID y borrador de diagramas de secuencia	Analiza este diseño y señala posibles violaciones SOLID. No reescribas todo. Convierte esta arquitectura validada en un	Hallazgos SOLID y borrador PlantUML de los dos flujos.	El borrador incluía un participante de notificaciones que no existe en el diseño.	Se eliminó el participante y se verificó que los participantes coincidieran con el diagrama de clases; se documentó IRegistroAudit

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
		borrador PlantUML sin agregar clases, reglas o relaciones que no aparezcan en la descripción.			oriaRepository con un único método agregar() (ADR-005).

Anexo C. Código Base en C# (Entregable 06)

Solo firmas e interfaces, sin implementación lógica. Se asume ID de pedido de tipo Guid (supuesto pendiente de validación).

C.1 Estructura de carpetas

```
SazonUrbano/
├── docs/
├── diagrams/
├── src/
│   ├── SazonUrbano.Domain/
│   │   ├── Common/            (RaizAgregado, IEventoDominio, IRepository,
IUnitOfWork)
│   │   ├── Pedidos/           (Pedido, ProductoPedido, Modificacion, EstadoPedido,
IPedidoRepository)
│   │   └── Inventario/        (Inventario, Ingrediente, AjusteMerma,
IInventarioRepository)
│   ├── SazonUrbano.Application/
│   │   ├── Common/            (Resultado, IDespachadorEventos)
│   │   ├── Pedidos/Commands/  (RegistrarPedidoCommand, RegistrarPedidoHandler)
│   │   ├── Pedidos/Queries/
│   │   └── Seguridad/         (ControlAccesoRBAC)
│   └── SazonUrbano.Infrastructure/
│       └── Persistence/       (UnitOfWork, PedidoRepository, InventarioRepository)
└── tests/
    └── SazonUrbano.Tests/
```

C.2 SazonUrbano.Domain

```
namespace SazonUrbano.Domain.Common;

public interface IEventoDominio { }

public abstract class RaizAgregado
{
    public Guid Id { get; protected set; }
    public IReadOnlyCollection<IEventoDominio> EventosDominio { get; }
    public void LimpiarEventos();
    protected void Registrar(IEventoDominio evento);
```

```
}

public interface IRepository<T> where T : RaizAgregado
{
    Task<T?> ObtenerPorIdAsync(Guid id, CancellationToken ct = default);
    Task GuardarAsync(T agregado, CancellationToken ct = default);
}

public interface IUnitOfWork
{
    Task ConfirmarAsync(CancellationToken ct = default);
    Task RevertirAsync(CancellationToken ct = default);
}
```

```
namespace SazonUrbano.Domain.Pedidos;

public enum EstadoPedido { Registrado, EnPreparacion, Listo, Entregado, Cancelado }

public sealed record PedidoRegistrado(Guid PedidoId) : IEventoDominio;
public sealed record PedidoEntregado(Guid PedidoId) : IEventoDominio;
public sealed record PedidoCancelado(Guid PedidoId, string Motivo, string UsuarioId,
DateTime FechaHora) : IEventoDominio;

public interface IPedidoRepository : IRepository<Pedido>
{
    Task<IReadOnlyList<Pedido>> ListarPendientesCocinaAsync(CancellationToken ct =
default);
}

// Aggregate Root operativo. Componentes: ProductoPedido (Entity), Modificacion
(Value Object).
public sealed class Pedido : RaizAgregado
{
    public EstadoPedido Estado { get; private set; }
    public IReadOnlyCollection<ProductoPedido> Productos { get; }

    private Pedido();

    public static Pedido Registrar(string canal, IEnumerable<ProductoPedido>
productos);
    public void IniciarPreparacion();
    public void MarcarListo();
    public void Entregar();
    public void Cancelar(string motivo, string usuarioId);
}

public sealed class ProductoPedido
{
    public Guid Id { get; }
    public int Cantidad { get; }
    public IReadOnlyCollection<Modificacion> Modificaciones { get; }
}

public sealed record Modificacion(string Descripcion);
```

C.3 SazonUrbano.Application

```
namespace SazonUrbano.Application.Common;

public sealed class Resultado<T>
{
    public bool EsExito { get; }
    public T? Valor { get; }
    public string? Error { get; }
    public static Resultado<T> Exito(T valor);
    public static Resultado<T> Fallo(string error);
}

public interface IDespachadorEventos
{
    Task DespacharAsync(IEnumerable<IEventoDominio> eventos, CancellationToken ct =
default);
}
```

```
namespace SazonUrbano.Application.Pedidos.Commands;

public sealed record RegistrarPedidoCommand(
    string Canal,
    IReadOnlyList<ProductoSolicitado> Productos,
    string UsuarioId);

public sealed record ProductoSolicitado(
    Guid ProductoId,
    int Cantidad,
    IReadOnlyList<string> Modificaciones);

// Orquestador delgado (ADR-003): no contiene reglas de negocio.
public sealed class RegistrarPedidoHandler
{
    private readonly IPedidoRepository _pedidos;
    private readonly IUnitOfWork _unitOfWork;
    private readonly IDespachadorEventos _despachador;

    public RegistrarPedidoHandler(
        IPedidoRepository pedidos,
        IUnitOfWork unitOfWork,
        IDespachadorEventos despachador);

    public Task<Resultado<Guid>> ManejarAsync(
        RegistrarPedidoCommand comando,
        CancellationToken ct = default);
}
```

Flujo esperado de ManejarAsync (ver Diagrama 1): crear Pedido mediante Pedido.Registrar, guardar con IPedidoRepository, confirmar con IUnitOfWork, despachar eventos con IDespachadorEventos y devolver Resultado.Exito con el ID del pedido. Ante un fallo al guardar, se revierte la transacción y no se despachan eventos.

Anexo D: Matriz de Trazabilidad Arquitectónica (Entregable 04)

Este apartado formaliza la relación directa entre los requerimientos aprobados en la Unidad 2 (Necesidades del Negocio, Reglas e Historias de Usuario) y los componentes de diseño arquitectónico y decisiones (ADRs) de la Unidad 3:

Unidad 2: Reglas / Historias / Requerimientos	**Unidad 3: Caso de Uso**	**Unidad 3: Componente Asignado**	**Unidad 3: Decisión (ADR)**	**Estado**
NB-03 (Información completa) **NB-06** (Conocer estado) **US-08** (Registrar pedido) **US-09** (Registrar modificaciones) **US-02** (Consultar disponibilidad)	CU-01: Registrar Pedido	**Domain Model:** Aggregate Root (Pedido) y Value Object (Modificacion). **CQRS:** RegistrarPedidoHandler y ConsultarDisponibilidadQuery.	ADR-001 ADR-003	Cubierto
US-01 (Consultar pedido cocina) **US-06** (Actualizar estado pedido) **US-07** (Consultar estado)	CU-02: Procesar Pedido	**Domain Model:** Transición de estados en la entidad Pedido. **CQRS:** ConsultarPedidoCocinaQuery y Handler de avance de estado.	ADR-001 ADR-002 ADR-003	Cubierto
NB-05 (Seguimiento cancelaciones) **US-10** (Registrar cancelación) **US-11** (Consultar cancelaciones)	CU-03: Cancelar Pedido	**Service Layer:** Interceptor/Middleware RBAC y CancelarPedidoHandler. **Infraestructura:** Tabla Append-only (LogAuditoria).	ADR-002 ADR-004 ADR-005	Cubierto

NB-08 (Control llevar/domicilio) **US-12** (Registrar entrega)	CU-04: Entregar Pedido	**Domain Model:** Evento PedidoEntregado. **Service Layer:** EntregarPedidoHandler y DescontarInventarioHandler.	ADR-002	Cubierto
NB-01 (Actualizar disponibilidad) **NB-02** (Evitar venta por falta de stock) **NB-07** (Reducir diferencias inv.) **US-03, US-04, US-05** (Inventario)	CU-05: Actualizar Inventario y Mermas	**Domain Model:** Aggregate Root (Inventario) y evento StockCriticoAlcanzado. **CQRS:** ActualizarStockCommand.	ADR-001	Cubierto
NB-09 (Información útil gerencia) **US-13** (Historial de modificaciones) **US-14** (Consultar ventas e inv.) **US-15** (Generar reportes)	Consultas Gerenciales (Lectura)	**Service Layer (CQRS):** ConsultarReportesQuery y lecturas sobre IRegistroAuditoriaRepository.	ADR-003 ADR-005	Cubierto

Dictamen de Cobertura

El equipo certifica que el 100% de las necesidades de negocio (NB) e historias de usuario (US) críticas definidas en el BRD de la Unidad 2 han sido mapeadas a los casos de uso operativos del restaurante Sazón Urbano. Asimismo, cada uno cuenta con un componente arquitectónico específico y su lógica ha sido resguardada mediante una decisión formalizada en los ADRs. Ninguna regla comercial queda expuesta en capas incorrectas, garantizando una implementación íntegra del patrón *Domain Model*, *CQRS* y *Consistencia Eventual*.